

Case 11: Negative — POST with Missing Fields

Intermediate

Verify how the API responds when required fields are missing from a POST request.

New concept: Negative testing on POST — incomplete payloads

API Endpoint

POST `https://jsonplaceholder.typicode.com/posts`

What You Will Learn

- Sending incomplete payloads intentionally
- Understanding how APIs handle missing fields
- Difference between client error (4xx) and success (2xx)
- Parametrizing multiple bad payloads in one test

Study Plan

Step 1 — What is a missing field test?

A well-behaved API should reject requests that are missing required fields. For example, a POST without a title or userId should return 400 Bad Request, not 201.

Step 2 — Build incomplete payloads

Create payloads that are missing one or more fields. Example: `{}` (empty), `{'title': 'only title'}` (missing body and userId), `{'userId': 1}` (missing title and body).

Step 3 — Parametrize the payloads

Think: what is the changing value between each test run — the URL or the payload? Use `parametrize` to test multiple bad payloads in one function.

Step 4 — What to assert

A real API would return 400. JSONPlaceholder is lenient and may return 201 even with missing fields. Think: how would you handle this with `xfail`?

Step 5 — Think about the loop

Each `parametrize` iteration gets one bad payload. Inside the test, do you need a for loop to check anything in the response, or is one status code assert enough?

Think Before You Code

Ask yourself these questions before writing a single line:

- What is the changing value between each test run — the URL or the payload?
- What type is each `parametrize` value here — a string, a number, or a dict?
- Do you need a for loop inside the test, or is one assert enough?
- How would you handle the fact that JSONPlaceholder accepts incomplete payloads?
- What HTTP status code should a real API return for a bad request?

Your Task

Write a parametrized pytest test that sends POST requests with incomplete payloads.

Test these payloads:

1. {} (completely empty)
2. {'title': 'only title'} (missing body and userId)
3. {'userId': 1} (missing title and body)

Before writing:

- What is the parametrize variable — the URL or the payload?
- What type is each parametrize value?
- Do you need a loop inside the test?

The test must verify:

1. Use parametrize for all 3 payloads
2. Mark each as xfail with reason 'JSONPlaceholder accepts incomplete payloads'
3. Assert status code is 400 for each

BONUS: Print the response body for each payload — what does JSONPlaceholder return?

Hint: The parametrize variable is the payload dict. `@pytest.mark.parametrize('payload', [{}, {'title': 'x'}, {'userId': 1}])`